

SIMATS ENGINEERING

ASSESSMENT TOOL 3: Modelling Comparative Analysis Assignment

COURSE NAME: ARTIFICIAL INTELLIGENCE **COURSE CODE:** MLA0101
AND EXPERT SYSTEMS

COURSE OUTCOME COVERED

CO 5: Construct rule-based expert systems using production rules, forward and backward chaining, and by distinguishing procedural and non-procedural paradigms across development stages.(BTL 3)

Assessment Tool Weightage: 40%

Total Marks: 100

Time: 90 Mins

No. of Questions: 1

Annexure A Modelling Comparative Analysis Assignment

Code of Conduct:

/ Kruthika Sre S / 192525360 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Kruthika Sre

Criterion	Weightage (%)	Marks Obtained
1. Problem Analysis and Domain Understanding	10%	
2. Domain Knowledge and Fact Identification	10%	
3. Production Rule Modelling	15%	
4. Propositional Logic Modelling	10%	
5. First-Order Logic Modelling	15%	
6. Prolog Modelling and Implementation	15%	
7. Forward and Backward Chaining Demonstration	10%	
8. Comparative Analysis of Modelling Approaches	10%	
9. Results, Discussion and Model Selection	3%	
10. Documentation, GitHub Repository and Presentation	2%	
Total	100%	

Signature of the Faculty

Total Marks Awarded

ANSWER ANY ONE OF THE FOLLOWING

S.No.	Question / Problem Statement
1	Automobile Fault Diagnosis: An automobile service center wants to identify vehicle faults based on symptoms such as engine overheating, starting failure, abnormal noise, low mileage, and warning indicators. Model the problem using production rules, propositional logic, first-order logic, and Prolog. Demonstrate forward and backward chaining and compare the approaches based on expressiveness, inference, complexity, and suitability.
2	Industrial Machine Fault Diagnosis: A manufacturing plant needs to identify machine faults using observations such as abnormal vibration, high temperature, unusual noise, pressure variation, and reduced production speed. Develop different knowledge representation models and compare their ability to represent machine components, relationships, rules, and diagnostic conclusions.
3	Cybersecurity Threat Detection: A security operations center wants to identify threats from repeated login failures, unusual login locations, privilege escalation, suspicious file access, and abnormal network traffic. Model the threat-detection problem using production rules, propositional logic, first-order logic, and Prolog, and compare forward and backward reasoning for deriving security conclusions.
4	Healthcare Diagnosis: A healthcare system needs to identify possible diseases from symptoms such as fever, cough, fatigue, breathing difficulty, and body pain. Construct comparative knowledge models using production rules, propositional logic, first-order logic, and Prolog. Analyze which representation provides better expressiveness and reasoning capability for diagnosis.
5	Agricultural Crop Disease Diagnosis: Farmers need assistance in identifying crop diseases based on leaf color, spots, wilting, soil conditions, humidity, and pest observations. Represent the agricultural knowledge using different modelling approaches and compare their effectiveness in representing relationships, applying rules, and deriving recommendations.
6	Banking Loan Eligibility: A bank wants to determine loan eligibility using income, credit score, employment status, existing loans, and repayment history. Model the decision-making process using production rules, propositional logic, first-order logic, and Prolog, and compare the reasoning capabilities

	of each approach.
--	-------------------

Structure of the Report:

S.No.	Report Section	Expected Content
1	Problem Statement & Objectives	Describe the selected real-world problem, domain, decision-making requirement, and objectives of the modelling exercise.
2	Domain Knowledge & Facts	Identify entities, attributes, facts, relationships, conditions, and conclusions relevant to the selected problem.
3	Production Rule Model	Represent the domain using IF–THEN production rules and explain how the rules derive conclusions.
4	Propositional Logic Model	Represent the same problem using propositions, logical connectives, and suitable inference statements.
5	First-Order Logic Model	Represent entities, relationships, properties, and rules using predicates and quantifiers.
6	Prolog Model & Implementation	Convert the knowledge into Prolog facts, rules, predicates, and queries. Include relevant code and outputs.
7	Forward & Backward Chaining	Demonstrate both reasoning mechanisms using selected facts/goals and provide the inference sequence.
8	Comparative Analysis (Table structure given)	Compare the models based on expressiveness, inference mechanism, complexity, scalability, flexibility, explainability, and suitability.
9	Results, Discussion & Recommended Model	Analyze the results and justify which approach is most appropriate for the selected real-world problem. Discuss limitations and possible improvements.
10	Conclusion, References & GitHub	Summarize findings, provide references, and include the GitHub repository link containing Prolog code, models, test cases, and supporting materials.

CYBERSECURITY THREAT DETECTION

Problem Statement

A Security Operations Center (SOC) needs to identify potential cybersecurity threats by analyzing security events such as repeated login failures, unusual login locations, privilege escalation, suspicious file access, and abnormal network traffic. The proposed system models these events using different knowledge representation and reasoning techniques to derive security conclusions and recommend appropriate responses.

Objectives

Objective	Description
Threat Identification	Identify potential security threats from observed events.
Knowledge Representation	Represent security knowledge using production rules, propositional logic, and first-order logic.
Prolog Implementation	Implement the threat-detection model using Prolog.
Logical Reasoning	Apply forward and backward reasoning to derive security conclusions.
Model Comparison	Compare the different representation and reasoning approaches.
Security Recommendation	Generate suitable conclusions such as suspicious activity or potential account compromise.

Domain Knowledge & Facts

The domain focuses on user authentication, access control, file activity, and network behavior.

Category	Example Facts
Authentication	Multiple failed login attempts
Login Location	Login from an unusual location
Privilege	Unexpected privilege escalation
File Access	Access to sensitive files
Network	Abnormal network traffic
Security Status	Suspicious activity, possible account compromise
Response	Investigate account, restrict access, monitor activity

Example scenario:

A user has repeated login failures followed by a successful login from an unusual location. The same user then accesses a sensitive file and performs a privilege escalation. These combined events can indicate potential account compromise.

Production Rule Model

The cybersecurity knowledge is represented using IF–THEN production rules.

Rule	Production Rule
R1	IF repeated login failures occur THEN mark the account as suspicious.
R2	IF login occurs from an unusual location THEN mark the login as suspicious.
R3	IF suspicious login AND repeated login failures occur THEN suspect account compromise.
R4	IF privilege escalation occurs unexpectedly THEN flag privilege escalation.
R5	IF suspicious file access occurs THEN flag suspicious file activity.
R6	IF abnormal network traffic occurs THEN flag abnormal network behavior.
R7	IF account compromise AND privilege escalation occur THEN generate a high-risk security alert.

This rule-based model allows the system to derive security conclusions from observed events.

Propositional Logic Model

In propositional logic, each security event is represented by a proposition.

Let:

Symbol	Meaning
F	Repeated login failures occurred
L	Login from unusual location occurred
P	Privilege escalation occurred
S	Suspicious file access occurred
N	Abnormal network traffic occurred
C	Account compromise is suspected
A	High-risk security alert is generated

The rules can be represented as:

R1:

$F \rightarrow \text{Suspicious_Login}$

R2:

$L \rightarrow \text{Suspicious_Login}$

R3:

$(F \wedge L) \rightarrow C$

R4:

$P \rightarrow \text{Privilege_Threat}$

R5:

$S \rightarrow \text{File_Threat}$

R6:

$N \rightarrow \text{Network_Threat}$

R7:

$(C \wedge P) \rightarrow A$

For example, if **F** and **L** are true, then:

$F \wedge L \rightarrow C$

Therefore, the system concludes that account compromise is suspected.

First-Order Logic Model

First-order logic provides greater flexibility by representing users and security events as objects and relationships.

Let $\text{User}(x)$ represent a user and the following predicates represent security events:

$\text{RepeatedLoginFailure}(x)$

$\text{UnusualLocation}(x)$

$\text{PrivilegeEscalation}(x)$

$\text{SuspiciousFileAccess}(x)$

$\text{AbnormalNetworkTraffic}(x)$

$\text{AccountCompromised}(x)$

$\text{HighRiskAlert}(x)$

The rules can be expressed as:

Rule 1:

$\forall x [\text{RepeatedLoginFailure}(x) \rightarrow \text{SuspiciousLogin}(x)]$

Rule 2:

$\forall x [\text{UnusualLocation}(x) \rightarrow \text{SuspiciousLogin}(x)]$

Rule 3:

$\forall x [(\text{RepeatedLoginFailure}(x) \wedge \text{UnusualLocation}(x)) \rightarrow \text{AccountCompromised}(x)]$

Rule 4:

$\forall x [\text{PrivilegeEscalation}(x) \rightarrow \text{PrivilegeThreat}(x)]$

Rule 5:

$\forall x [(\text{AccountCompromised}(x) \wedge \text{PrivilegeEscalation}(x)) \rightarrow \text{HighRiskAlert}(x)]$

For example, if:

RepeatedLoginFailure(User1)

UnusualLocation(User1)

then the inference system derives:

AccountCompromised(User1)

Prolog Model & Implementation

The knowledge can be implemented in SWI-Prolog using facts and rules.

% Security event facts

repeated_login_failure(user1).

unusual_location(user1).

privilege_escalation(user1).

suspicious_file_access(user1).

abnormal_network_traffic(user2).

% Production rules

suspicious_login(User) :-

 repeated_login_failure(User).

suspicious_login(User) :-

 unusual_location(User)

account_compromised(User) :-

 repeated_login_failure(User),

 unusual_location(User).

privilege_threat(User) :-

 privilege_escalation(User).

file_threat(User) :-

 suspicious_file_access(User).

network_threat(User) :-

 abnormal_network_traffic(User).

high_risk_alert(User) :-

 account_compromised(User),

 privilege_escalation(User).

Example queries:

?- suspicious_login(user1).

true.

?- account_compromised(user1).

true.

?- high_risk_alert(user1).

true.

Thus, Prolog automatically matches the facts with the appropriate rules and derives the security conclusions.

Forward & Backward Chaining

Forward Chaining:

Forward chaining starts with known security events and moves toward conclusions.

Repeated Login Failure

+

Unusual Location

↓

Suspicious Login

↓

Account Compromise Suspected

+

Privilege Escalation

↓

High-Risk Alert

Backward Chaining:

Backward chaining starts with a security goal and works backward to determine whether the required conditions are satisfied.

Goal: High-Risk Alert

↓

Requires Account Compromise

+

Privilege Escalation

↓

Account Compromise requires:

Repeated Login Failure +

Unusual Location

↓

Check available facts



Conclusion: High-Risk Alert

Comparative Analysis

Feature	Production Rules	Propositional Logic	First-Order Logic	Prolog
Representation	IF-THEN rules	Propositions	Predicates and variables	Facts and rules
Variables	Limited	No	Yes	Yes
Rule Matching	Yes	Logical inference	Logical inference	Automated
Forward Reasoning	Possible	Possible	Possible	Supported
Backward Reasoning	Possible	Possible	Possible	Native
Scalability	Moderate	Limited for complex domains	High	High
Flexibility	Moderate	Low	High	High
Implementation	Rule-based	Logic-based	Logic-based	Executable logic
Suitability	Easy to understand	Simple relationships	Complex relationships	Practical expert systems

Results, Discussion & Recommended Model

The system successfully represents cybersecurity events using all four approaches. Production rules provide an intuitive representation of security policies, while propositional logic is suitable for simple Boolean relationships. First-order logic provides greater expressiveness by allowing variables and relationships between users and security events.

Prolog provides a practical implementation because it combines facts, rules, pattern matching, and automated logical inference. It also naturally supports backward chaining, making it particularly suitable for querying whether a specific security threat can be derived from the available evidence.

Recommended Model

Prolog with production rules and backward reasoning is recommended for the proposed expert system because it provides a clear representation of security knowledge and efficient logical query-based reasoning. Forward reasoning can additionally be used when continuously processing incoming security events.

Conclusion, References & GitHub

Conclusion:

The Cybersecurity Threat Detection Expert System demonstrates how knowledge representation and logical reasoning can be applied to identify potential security threats. Repeated login failures, unusual login locations, privilege escalation, suspicious file access, and abnormal network traffic can be represented as facts and rules. The comparison shows that Prolog provides an effective implementation platform because of its declarative nature, rule matching, and automated inference capabilities. Combining forward and backward reasoning allows the system to derive meaningful security conclusions from observed events.

References:

1. S. Russell and P. Norvig, Artificial Intelligence: A Modern Approach, Pearson.
2. I. Bratko, Prolog Programming for Artificial Intelligence, Pearson.
3. SWI-Prolog Documentation, SWI-Prolog Reference Manual.
4. W. Stallings, Computer Security: Principles and Practice, Pearson.

Compulsory Comparative Table:

Parameter	Production Rules	Propositional Logic	First-Order Logic	Prolog
Knowledge Representation	IF–THEN rules	Boolean propositions	Predicates, variables, and quantifiers	Facts and rules
Expressiveness	Moderate	Low	High	High
Rule Representation	Direct IF–THEN format	Logical implications	Predicate-based rules	Natural rule-based syntax
Inference Mechanism	Rule matching	Logical deduction	Predicate-based inference	Automated logical inference
Forward Chaining	Supported	Possible	Supported	Can be implemented
Backward Chaining	Supported	Possible	Supported	Naturally supported
Handling Relationships	Limited	Very limited	Excellent	Excellent
Scalability	Moderate	Low	High	High
Explainability	High	High	High	High
Ease of Implementation	Easy	Easy	Moderate	Easy to moderate
Suitability for the Problem	High	Moderate	Very High	Very High

EVALUATION RUBRICS

Criterion	Weightage (%)	Excellent	Good	Average	Poor
1. Problem Analysis and Domain Understanding	10%	9–10% – Clearly analyzes the real-world problem, domain entities, constraints, facts, and expected outcomes.	7–8% – Good understanding with minor omissions.	5–6% – Basic understanding with limited analysis.	0–4% – Problem is unclear or poorly understood.
2. Domain Knowledge and Fact Identification	10%	9–10% – Identifies comprehensive and relevant facts, entities, relationships, conditions, and domain knowledge.	7–8% – Identifies most relevant knowledge with minor gaps.	5–6% – Basic facts and entities are identified.	0–4% – Important domain knowledge is missing or incorrect.
3. Production Rule Modelling	15%	13–15% – Develops accurate, complete, consistent IF–THEN production rules that effectively represent the problem.	10–12% – Rules are appropriate with minor errors or omissions.	7–9% – Basic rules are developed but several are incomplete or unclear.	0–6% – Rules are largely incorrect, incomplete, or inappropriate.
4. Propositional Logic Modelling	10%	9–10% – Correctly represents domain knowledge using propositions, logical connectives, and valid inference statements.	7–8% – Good propositional representation with minor errors.	5–6% – Basic representation with limited logical formulation.	0–4% – Representation is incorrect or incomplete.
5. First-Order Logic Modelling	15%	13–15% – Accurately models entities, relationships, properties, variables, predicates, and quantifiers with valid FOL statements.	10–12% – Good FOL model with minor errors or omissions.	7–9% – Basic FOL representation with limited relationships or incorrect usage of some constructs.	0–6% – FOL model is largely incorrect or missing.

6. Prolog Modelling and Implementation	15%	13–15% – Successfully converts the knowledge model into Prolog facts, rules, predicates, and queries with correct outputs.	10–12% – Functional Prolog implementation with minor technical issues.	7–9% – Basic implementation works but has limited functionality.	0–6% – Implementation is incomplete or non-functional.
7. Forward and Backward Chaining Demonstration	10%	9–10% – Clearly demonstrates both forward and backward chaining with correct inference sequences and conclusions.	7–8% – Both mechanisms are demonstrated with minor gaps.	5–6% – Basic demonstration with limited reasoning explanation.	0–4% – Chaining is absent or incorrectly demonstrated.
8. Comparative Analysis of Modelling Approaches	10%	9–10% – Provides a detailed and insightful comparison based on expressiveness, inference, complexity, scalability, flexibility, and explainability.	7–8% – Provides a meaningful comparison using appropriate parameters.	5–6% – Basic comparison with limited criteria or analysis.	0–4% – Little or no meaningful comparison.
9. Results, Discussion and Model Selection	3%	3% – Clearly analyzes results and convincingly justifies the most suitable approach for the selected problem.	2% – Provides good analysis and reasonable justification.	1% – Basic discussion with limited justification.	0% – No meaningful analysis or justification.
10. Documentation, GitHub Repository and Presentation	2%	2% – Report is complete and well structured, GitHub repository is organized, and presentation is professional.	1.5% – Good documentation and presentation with minor issues.	1% – Basic documentation with some omissions.	0% – Poor documentation, missing repository, or inadequate presentation.
Total	100%				